

第一、二章

软件工程定义

1. **软件工程**：应用系统的、规范的可量化的方法来开发、运行和维护软件，即将管理应用到软件中。以及对上述方法的研究。（它具有解决实际问题的动机、应用科学知识指导工程活动、以成本效益比为基本条件、构建机器或事物、以服务人为最终目的）

软件工程发展

- **从1950-2000之间的特点**
 - 1950s：科学计算；以机器为中心进行编程；像生产硬件一样生产软件。
 - 1960s：业务应用(批量数据处理和事物计算)；软件不同于硬件；用软件工艺的方式生产软件。
 - 1970s：结构化方法；瀑布模型；强调规则和纪律。它们奠定了软件工程的基础，是后续年代软件工程发展的支撑。
 - 1980s：追求生产力最大化；现代结构化方法/面向对象编程广泛应用；重视过程的作用。
 - 1990s：企业为中心的大规模软件系统开发；追求快速开发、可变更性和用户价值；web应用出现
 - 2000s：大规模web应用；大量面向大众的web产品；追求快速开发、可变更性、用户价值和创新。

	1950s	1960s	1970s	1980s	1990s	2000s
硬件变化	研究用大型机	商业大型机	商业微型机	个人电脑PC	Intranet	Internet 小型设备
系统软件变化		BIOS	操作系统 数据库管理系统	图形化操作系统	网络操作系统 中间件平台	面向Web的中间件平台 嵌入式操作系统
问题类型	科学计算	从科学计算 转向业务应用	业务应用	可靠的“生产能力” 业务应用	规模增长 复杂软件系统	大规模Web应用 大众Web产品
难点	硬件性能	程序正确性、可靠性	复杂系统的复杂性	数量增长 生产效率	规模大幅增长 复杂性、时间压力 可变性、用户价值	转向Web 同1990s 创新
软件开发技术	无	从科学计算 转向业务应用	工艺方法	程序的“正确性” 可靠性与生产效率	结构化编程	
			编程→设计→分析 控制复杂系统的复杂性	结构化分析与设计	由程序构建 转向结构构建	现代结构化方法
				控制复杂系统的复杂性 生产力追求	面向对象编程	编程→设计→分析 面向对象方法
					基于复用的大规模软件开发技术	基于复用的大规模软件开发技术
					基于复用的大规模软件开发技术	基于复用的大规模软件开发技术
软件开发技术	• • •				大规模软件的复杂性 时间压力	基于复用的大规模软件开发技术
					大规模软件的复杂性 可变性 用户价值	软件体系结构 软件需求工程 人机交互
					Web应用出现	Web技术
						职业成熟 市场分化
						分领域的软件开发技术
软件开发过程	硬件工程	软件≠硬件	构建-修复	生产纪律 (工程生产)	瀑布模型	纪律与规则下的灵活性 追求生产力与产品质量
					过程质量影响产品质量	迭代、并发、快速的增量或演化模型
					过程评价SW-CMM	发展
					重规划、研究、工具支持	计算机辅助软件工程CASE
					黑客文化→自由软件	开源软件
软件开发过程					全球化	外包软件

- 时间分割：保证全天候有人在
- 产品质量的降低
- 虚假的最后期限
- 小圈子管理

质量保障

- **质量保障有哪些措施？**

- **需求开发**：需求评审和需求度量；
- **体系结构**：体系结构评审和集成测试（持续集成）；
- **详细设计**：详细设计评审、设计度量和集成测试（持续集成）；
- **构造阶段**：代码评审、代码度量、测试（测试驱动和持续集成）；
- **测试阶段**：测试、测试度量。
- 要及时的根据保障计划进行质量验证，质量验证的方法主要有评审、测试和质量度量三种。

- **配置管理有哪些活动？**

配置项：置于软件配置管理之下的软件配置的各种有关项目，包括各类管理文档、评审记录与文档、软件文档、源码及其可执行码、运行所需的系统软件和支持软件以及有关数据等

- **标识配置项版本管理**：确定应该被保留的部分，并且给予他们确定标识，包含配置项的特征，包括生产者、基线建立时间、使用者等。
- **版本管理**：极其重要
- **变更控制**：变更请求表单，教材61页
- **配置审计**：验证配置项的完整性、正确性、一致性和可追踪性。
- **状态报告**：反映当前的配置状态。
- **软件发布管理**：将配置项发布到开发活动之外，例如发布给客户。

第五章 (重点)

需求定义

- 需求是用户为了解决问题或者达到某些目标所需要的条件或能力
- 系统或系统部件为了满足合同、标准、规范或其他正式文档而需要具备条件和能力
- 是对上述的一个条件或一个能力的文档化描述

- **需求是一种期望**，“在...时，系统应该...”、“用户可以通过系统...”，问题是糟糕的一面，而需求是理想的一面

问题域（现实世界运行规律的反映，需求的产生域与解决地，要尊重问题域）、规格说明（软件产品的方案描述，描述运行机制，不是需求但实现需求，不是问题域但与问题域互动，关注对外交互）

需求的三个层次

考察方式：给出一个实例，给出其三个层次需求的例子；对给定的需求示例，判断其层次。例如课程实验/ATM/图书管理.....

1. 业务需求（目标）

- 业务需求是目标，是解决方案与系统特性
- 业务需求是高层次的解决方案和系统特性（不涉及到细节，限定了系统的**范围**）、系统开发分战略出发点、高层次需求，描述为什么要开发系统

2. 用户需求（做什么）

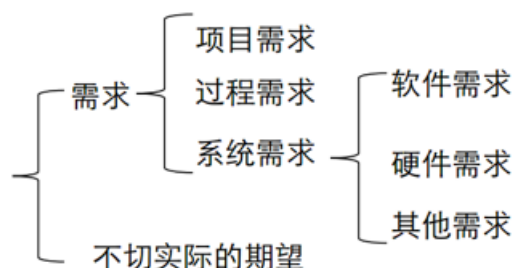
- 用户需求是任务，是问题域知识
- 用户需求是描述系统能够帮用户做什么，需要了解问题域知识，是模糊不清晰的、多特性多逻辑混杂的

3. 系统需求（怎么做）

- 系统需求描述了开发人员需要实现什么，将用户需求转换为系统需求是一个复杂的过程
 - 需要分析问题域及其特性，建立系统的知识模型
 - 然后将用户需求部署到系统模型中，定义系统行为使之联合实现用户需求
 - 该过程就是需求工程中最重要需求分析活动，又称**建模与分析活动**

需求的类型

需求谱系分类



对给定的实例，给出其不同类型的需求例子；对给定的需求示例，判断其类型

- 例如课程实验/ATM/图书管理.....
- 譬如几个非功能需求

1. 功能需求

- **功能需求主要表现为系统和环境之间的行为交互。**
- 和系统主要工作相关的需求，即在不考虑物理约束的情况下，用户希望系统能够执行的活动，帮助用户完成任务

2. 数据需求（功能需求的补充）

- 功能需求的补充，在功能需求部分明确定义了相关的数据结构，就不需要再进行数据需求的定义
- 需求在数据库、文件或者其他介质中存储的数据描述：数据信息、使用频率、可访问性要求、完整性约束、数据保持要求

3. 性能需求

- 系统整体或系统组成部分应该拥有的性能特征，例如CPU使用率，内存使用率等
- 包括：速度、容量、吞吐量、负载（并发工作量）、实时性

4. 质量属性

- 系统为了满足**规定的及隐含**的所有要求而需要具备的要素称为质量
- 包括：可靠性、可用性、安全性、可维护性、可移植性、易用性（往往包含形容词和副词）

5. 对外接口

- 系统和环境中其他系统之间需要建立的接口，包括硬件接口、软件接口、数据库接口等等
- 包括规定：接口用途、输入输出、数据格式、命令格式、异常处理要求

6. 约束

- 进行系统构造时需要遵守的约束，例如**编程语言、硬件设施**等
- 包括：系统开发与运行环境、问题域标准、商业规则

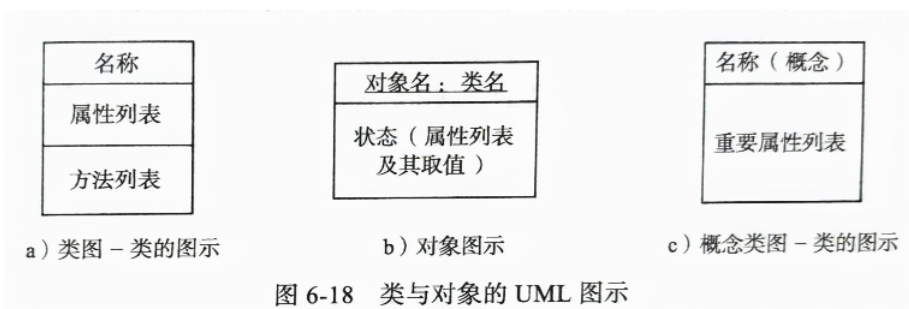
第六章

用例图

- 什么是用例：是在系统（或子系统或类）和外部对象的交互当中执行的**行为序列**的描述，包括**各种不同的序列和错误的序列**，他们能够联合提供一种**有价值的服务**
 - 图例：
 - 用例：椭圆
 - 参与者：小人
 - 关系：简单的一条直线，包括相关、泛化关系、包含关系和继承关系
 - 系统边界：一个框
 - 建立步骤：
 1. 目标分析与解决方向的确定——我们需要实现什么
 2. 寻找参与者；寻找用例
 3. 细化用例——判断一个用例描述的是不是一个**业务事件**，由**一个用户**发起，并在**一段连续时间**内完成，可以**增加业务价值**
 - 不要细化过小，导致其失去独立业务价值
 - 不要把同一业务目标用例细化为两个用例
 - 不要将没有业务价值(而是技术实现需要)的内容作为用例:常见的错误有**登录**(应该描述为安全性质量需求)、“数据验证/输入/输出数据检查”(应该描述为数据需求或者业务规则)、“连接数据库”(属性软件内部实现而不是需求)、网络传输等。
 - 不要将单个步骤细化为用例
 - 不要将片面的一个方面细化为用例
-

概念类图

- 指“领域模型”，是面向对象分析方法的核心：描述类与类之间的关系
- 注意与设计类图做区分：概念类图关注系统与外界的交互，而不是软件系统的内部构造机制；类型、方法、可见性等复杂软件构造细节不会出现在概念类图中
- 组成
 - 对象：对具体问题域的抽象（标识符+状态+行为）
 - 类：对对象进行分类，对象是类的实例；概念类显式地描述自己一些重要属性，不包含明确方法



- 链接：对象间相互协作的联系，描述对象之间的物理或业务联系
- 关联：用户间的关系，是链接的抽象
 - 聚合（整体和部分）和组合（+完全管理职责+所属唯一性）

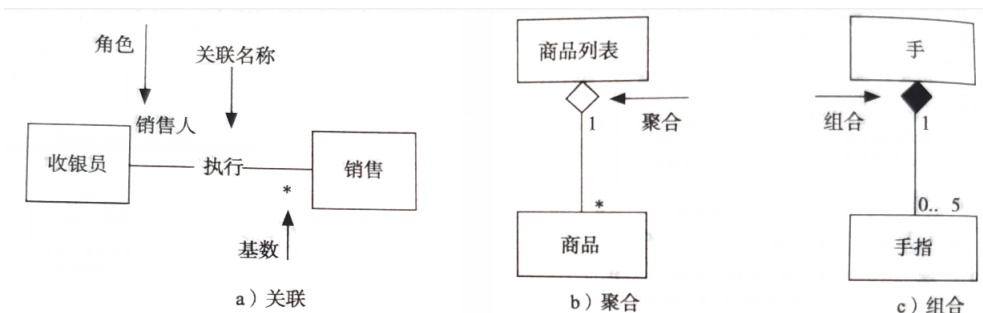


图 6-19 关联的 UML 图示

- 继承

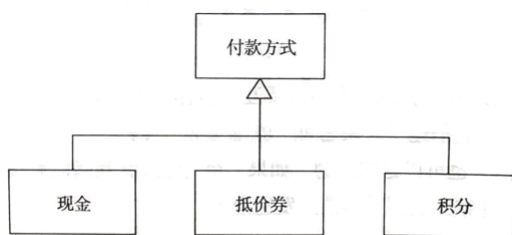


图 6-20 继承的 UML 图示

建立步骤

1. 识别候选类（交互可能设计的对象与类，名词筛选法）
2. 筛选候选类，确定概念类（标准，该对象的状态和行为是否必要）
 - (状态+行为)既需要维持一定的状态，又需要依据状态表现一定的行为
 - 确定为一个概念类
 - (仅状态)如只需要维护状态，不需要表现行为
 - 确定是不是其他概念类的属性
 - (仅行为)不需要维护状态，却需要表现行为
 - 首先要重新审视需求是否有遗漏，因为没有状态支持的对象无法表现行为
 - 如果确定没有需求的遗漏，就需要剔除该候选类，并将行为转交给具备状态支持能力的其他概念类
 - (无状态行为)既不需要维护状态，又不需要表现行为:废弃

3. 识别关联（文本中提取出"名词 + 动词 + 名词"的结构）

4. 识别重要属性

5. （先画关联关系，再添加类的属性）

系统顺序图

- 顺序图显示了对象之间的交互，以产生一些特定的系统行为。通常是描述单个用例的场景

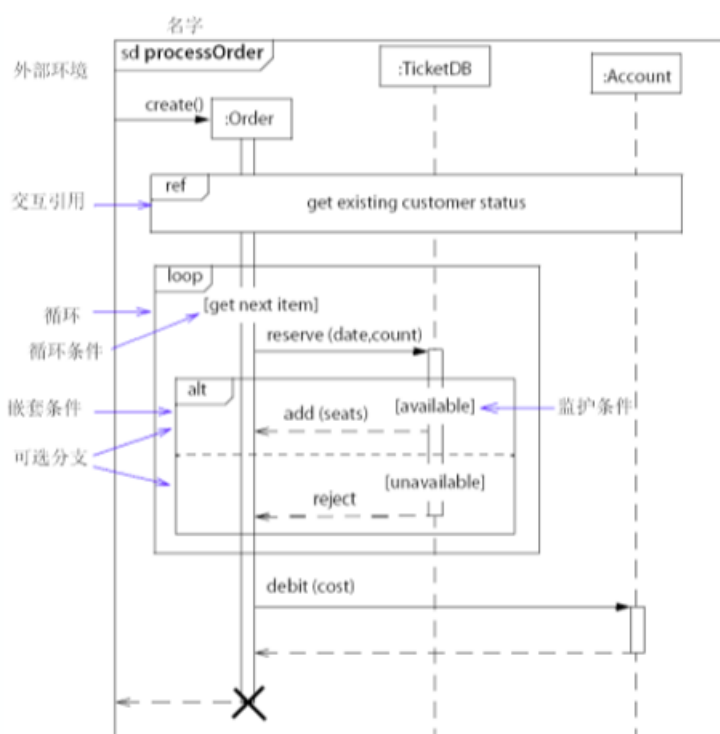


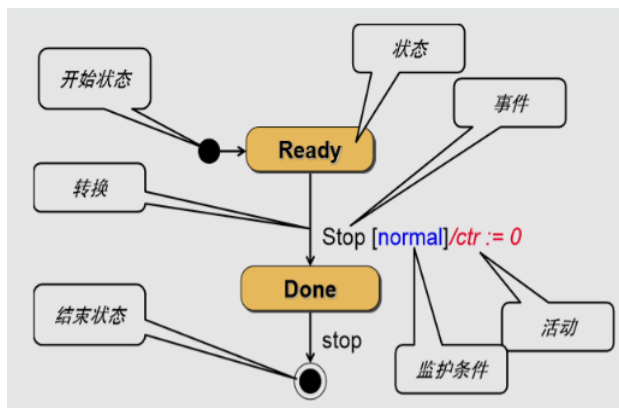
图 14-20. 特化图示示例，源自 [Rumbaugh2004]

——> 同步消息 -.-> 异步消息 > 返回消息

- alt一定要选（多选一）：注意，每一种可选分支之间要用虚线分割，而且在表示执行态的圆柱上面要写监护条件，放在[]里面。
- opt一定要选（选择0或者1）
- loop：表示循环，在旁边使用[]书写循环条件
- 建立步骤
 1. 确定上下文环境
 2. 根据用例描述找到交互对象
 3. 按照用例描述中的流程顺序逐步添加消息

状态图

- 状态是一组可观察的情况，描述了一个系统在给定时间里的行为；从一个状态到另一个状态的转换
- 事件：使系统表现出某种在那个可以预测的行为形式的事件
- 行为：由于过渡而发生的过程



- 建立步骤
 1. 确定上下文环境
 - 首先要搞清楚主体，常见的主体有：类、用例、多个用例和整个系统
 2. 识别状态
 - 注意开始状态和结束状态
 3. 建立状态转换
 4. 补充详细信息，完善状态图

几张图的绘制可以看一下书本的例子，比较详尽，搞清楚为什么

这里再记录依稀阿济格比较有意思的点，就是我们为什么需要这几张图？

1. 概念类图有助于我们发现信息的使用是否准确，是否遗漏了重要内容
2. 系统顺序图有助于发现交互性的缺失
3. 状态图有助于我们发现页面的跳转

第七章

为什么需要需求规格说明？结合实验进行说明

1. **方便交流**：在软件开发过程汇总，子任务与人员之间存在错综复杂的关系，存在大量的沟通和交流，所以需要编写软件开发中不同类型的文档，每种文档都是项

目中需要广泛交流的内容。**因为软件需求需要进行广泛交流，所以把需求文档化。**

2. **跟踪与度量**：需求规格说明是在软件产品的角度以系统级需求列表的方式描述软件系统解决方案，书写需求规格说明，可以**建立管理的基线，方便任务分配，指定工作计划，进行跟踪和度量。**
3. **过程管理**：在实验中，需求规格的重要性不只体现在结果上，还包括中间过程，在**书写需求规格的过程中，才真正把问题域的问题和分析模型成果转换为系统级需求**，方便小组成员**真正明确需求**，同时也包含一部分的需求在发现的完整化

对给定的需求示例/需求规格说明片段，判定并修正其错误

很有意思的一部分

- 技术文档协作要点
 - 简洁：平铺直叙，减少复杂长句、形容词和副词
 - 精确：避免歧义表述，尤其一些表示程度的需要量化
 - 易读（查询）：引言、目录、索引等；使用系统化的方式组织（语句格式、表格、编号等）
 - 易修改：易读性的措施 + 使用引用代替重复
- 需求书写要点：
 - 使用用户术语（避免专业词汇）
 - 可验证（给出量化标准，比如查询页面使用友好——完成查询操作点击次数不超过五次）
 - 可行性
- 需求规格说明文档书写要点
 - 充分利用标准的文档模板，保持所以内容位置得当
 - 保持文档内的需求集具有完备性和一致性
 - 为需求划分优先级

一些歧义词语

歧义词汇	改进方法
可接受的、足够的	具体定义可接受的内容，说明系统怎样判断“可接受”或“足够”
依赖	描述依赖的原因，数据依赖？服务依赖？还是资源依赖？等等
有效的	明确“有效”所意味的具体实际情况
快的、迅速的	明确指定系统在时间或速度上可接受的最小值
灵活的	描述系统为了响应条件变化或需求变化而可能发生的变更方式
改进的、更好的、优越的	定量说明在一个专门的功能领域内，充分改进的程度和效果
包括但不限于、等等、诸如	应该列举所有的可能性，否则就无法进行设计和测试
最大化、最小化、最优	说明对某些参数所能接受的最大值和最小值
一般情况下、理想情况下	需要增加描述系统在异常和非理想情况下的行为
可选择地	具体说明是系统选择、用户选择还是开发人员选择
合理的、必要的、适当	明确怎样判断合理、必要和适当
健壮的	显式定义系统如何处理异常和如何响应预料之外的操作
不应该	试着以肯定的方式陈述需求，描述系统应该做什么
最新技术水平的	定义其具体含义，即“最新技术水平”意味什么
充分的	说明“充分”具体包括哪些内容
支持、允许	精确地定义系统的功能，这些功能组合起来支持某些能力
用户友好的、简单的、容易的	描述系统特性，用这些特性说明词汇所代表的用户期望的实质

对给定的需求示例，设计功能测试用例

结合测试方法

- 对给定的需求规格说明片段，找出并修正其错误

ID	输入				预期输出
	商品信息	特价	赠品	支付	
TUS1-1	无商品	无	无	无	不做任何处理，关闭销售任务
TUS1-2	商品1 (1、1(双)、35) 商品2 (2、1(双)、50)	无	无		
TUS1-3	商品1 (1、1(双)、35) 商品2 (2、1(双)、50)	无	无	85	无找零，系统行为满足后置条件
TUS1-4	商品1 (1、1(双)、35) 商品2 (2、1(双)、50)	商品1特价20		100	找零30，系统行为满足后置条件
TUS1-5	商品1 (1、1(双)、35) 商品2 (2、1(双)、50)	总额特价50以上0.8折		100	找零31，系统行为满足后置条件
TUS1-6	商品1 (1、1(双)、35) 商品2 (2、1(双)、60)	商品1特价20 总额特价50以上0.8折		100	找零32，系统行为满足后置条件

第八章（思想理解即可）

软件设计

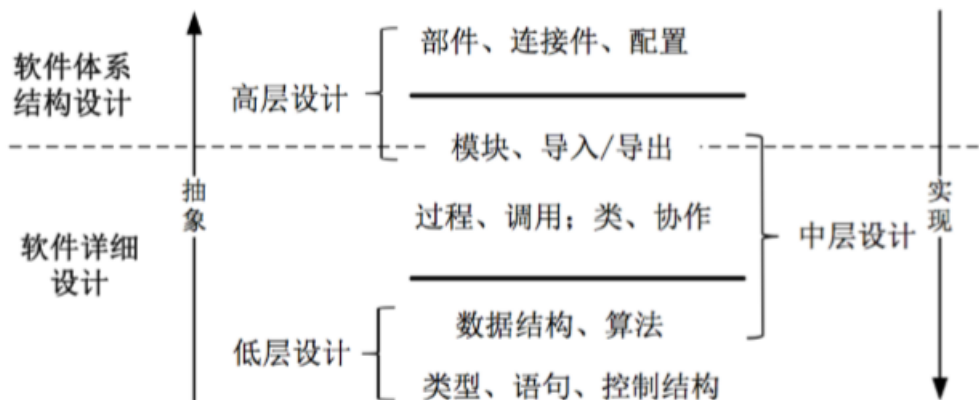
- 软件设计是指关于软件对象的设计，是一种设计活动。软件设计既指软件对象实现的**规格说明**（名词），又指这个**规格说明产生的过程**（动词）
 - 软件设计活动以需求开发的制品（需求规格说明和分析模型）为基础，**构建软件设计方案描述和原型**，为后期的构造活动提供规划或蓝图。
 - 软件设计**兼具工程性和艺术性**，由于软件系统的可变性，软件设计具有**演化性**，也因为软件设计的过程实际上就是一系列决策发生的过程，软件设计具有**决策性**
-

软件设计的核心思想

- **分解**：横向上将系统分割为几个相对简单的子系统与子系统之间的关系
 - **抽象**：在纵向上聚焦个子系统的接口（这里的接口与实现相对），可以分离接口和实现，使得人们更好的关注软件系统本质，降低复杂度
-

软件工程设计三个层次，各层次主要思想

- **高层设计**：基于反映软件高层抽象的构件设计，描述系统的高层结构、关注点和设计决策。
 1. **部件**承载了系统主要的**计算与状态**
 2. **连接件**承载部件之间的**交互**
 3. 部件与连接件都是抽象的类型定义(就像类定义)，它们的实例(就像类的对象实例)组织构成软件系统的整体结构，**配置**将它们的实例连接起来
- **中层设计**：更加关注组成构件的模块的设计、导入/导出、过程之间调用关系或者类之间的协作，模块划分**隐藏**一些程序片段(数据结构+算法)的细节，暴露接口于外界
- **低层设计**：深入模块和类的内部，关注具体的数据结构、算法、类型、语句和控制结构等。



软件设计的分层

第九、十章

体系结构的概念

- 软件体系结构是由部件、连接件、配置组成的，“一个软件系统的体系结构规定了系统的计算部件和部件之间的交互”
- 部件是软件体系结构的基本组合单元之一，**承载系统的主要功能**，包括处理和数据
 - 原始部件——可以直接被实现为相应的软件实现机制
 - 复合部件——由更细粒度的部件和连接件通过局部配置组织起来
- 连接件是软件体系结构的另一个基本组成单元，定义了**部件间的交互**，是**连接的抽象表示**
 - 原始连接件、复合连接件
 - 隐式实现、显式实现
- 配置是对“形式”的发展，定义了“**部件**”以及“**连接件**”之间的**关联方式**，将它们**组织**成系统的总体结构
 - ADL语言，描述软件体系结构的形式化模型语言。
 - 并不是直接指定部件和连接件的关系，而利用配置，好处是可以实现一次定义多次使用，可以随时变化

体系结构的风格的优缺点

四种风格与它们的优缺点

1. 主程序/子程序

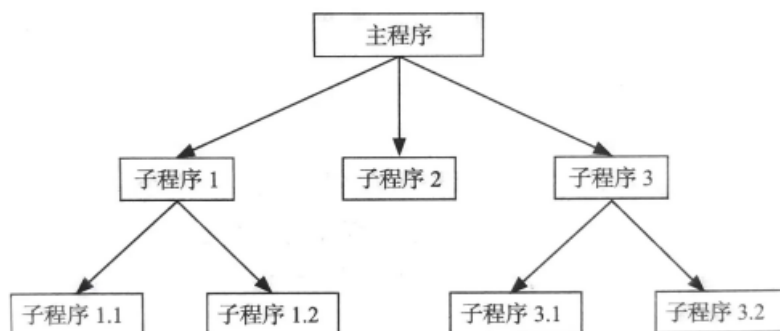


图 9-4 主程序 / 子程序风格

- 描述：将系统组织成层级结构，包括一个主程序和一系列子程序。主程序是系统的控制器，负责调度各子程序的执行。各子程序又是一个局部控制器，负责调度其子子程序的执行
- 设计决策与约束：
 - 基于声明-使用（程序调用）管理建立连接件
 - 上层可以使用下层部件，不允许逆向调用
 - 系统是单线程运行，控制权的转移
- 优点：
 - 流程清晰，易于理解——符合功能分解和分而治之的思想
 - 强控制性——控制权的转移
- 缺点：
 - 程序调用时一种强耦合的连接方式，非常依赖接口，使得系统难以修改和复用
 - 程序间调用方式限制了各部件之间的数据交互，可能会使不同部件使用隐式共享数据交流，产生不必要的公共耦合

2. 面向对象式

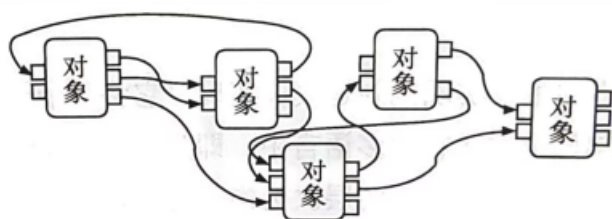


图 9-5 面向对象式风格

- 描述：将系统组织为多个独立的对象，每个对象封装其内部的数据，并基于数据对外提供服务，不同对象通过协作机制共同完成系统任务

- 设计决策与约束
 - 用信息内聚的标准为系统建立对象部件，每个对象基于内部数据提供对外服务接口，并隐藏内部数据展示
 - 基于方法调用的机制建立连接件
 - 每个对象负责维护自身数据的一致性与完整性
 - 每个对象是自洽的，对象之间是对等的
- 优点：
 - 内部实现可以修性——不影响外部
 - 易开发、易理解、易复用的结构组织
- 缺点：
 - 接口的耦合性——方法调用的连接机制带来的
 - 标识的耦合性——对象间交互必须要知道其他对象的标识
 - 副作用——面向对象带来的修改丢失、对象重入等等

3. 分层

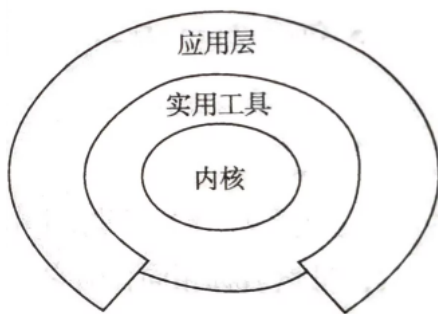


图 9-6 分层风格

- 描述：层次结构，每个层次被建立为一个部件，通过程序调用的方式进行连接
- 设计决策和约束：
 - 低层到高层，部件的抽象层次逐渐提升，下层为相邻上层提供服务，跨层次的连接是禁止的
 - 层次间遵守交互协议，逆向连接是禁止的
- 优点：
 - 设计机制清晰，易于理解
 - 支持并行开发——交互协议成熟稳定标准化
 - 更好的复用性和内部可修改性——只要遵守协议，内部实现更为自由，不同层次的部件能够相互替换
- 缺点：
 - 交互协议难以修改
 - 性能损失——跨层调用的禁止
 - 难以确定的层次数量和粒度

4. MVC

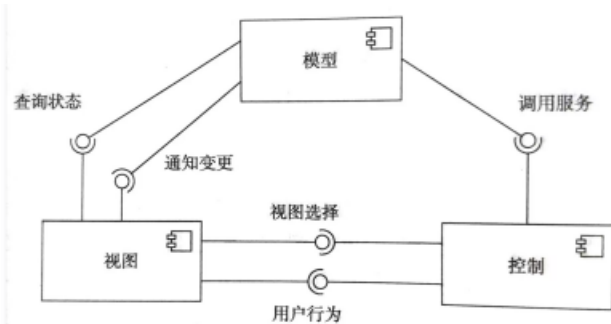


图 9-7 模型-视图-控制风格

MVC: moudle模型 + view视图 + control控制

- 描述：
 - 将系统功能组织为模型、视图、控制三个部件。
 - 模型封装系统的数据和状态信息，实现业务逻辑，对外提供数据服务和执行逻辑业务
 - 视图封装了有用户交互，提供业务展示，接收用户行为
 - 控制封装了系统的控制逻辑，根据用户行为调用需要执行的业务逻辑和数据更新，并根据执行之后的系统状态决定后续的业务展示
- 设计决策与约束：
 - 模型、视图、控制分别是关于业务逻辑、表现和控制的三种不同内容抽象
 - 如果视图需要持续地显示某个数据的状态,那么它首先需要在模型中注册对该数据的兴趣。如果该数据状态发生了变更,模型会主动通知视图,然后再由视图查询数据的更新 情况。
 - 视图只能使用模型的数据查询服务,只有控制部件可以调用可能修改模型状态的程序。
 - 用户行为虽然由视图发起,但是必须转交给控制部件处理。对接收到的用户行为, 控制部件可能会执行两种处理中的一种或两种:调用模型的服务,执行业务逻辑;提供下一个业务展现。
 - 模型部件相对独立,既不依赖于视图,也不依赖于控制。虽然模型与视图之间存在一个"通知变更"的连接,但该连接的交互协议是非常稳定的,可以认为是非常弱的依赖。
- 优点：
 - 易开发性——分别抽象了业务逻辑，表现和控制机制，清晰，易于开发
 - 视图和控制的可修改性——模型是相对独立的，不会受到影响
 - 适宜于网络系统开发的特征——MVC 不仅允许视图和控制的可修改性，而且其对业务逻辑、表现和控制的分离使得一个模型可以同时建立并保持**多个视图**，这非常适用于网络系统开发)
- 缺点：
 - 复杂性——不利于理解任务的实现

- 模型修改困难——视图和控制都依赖于模型

体系结构设计的过程

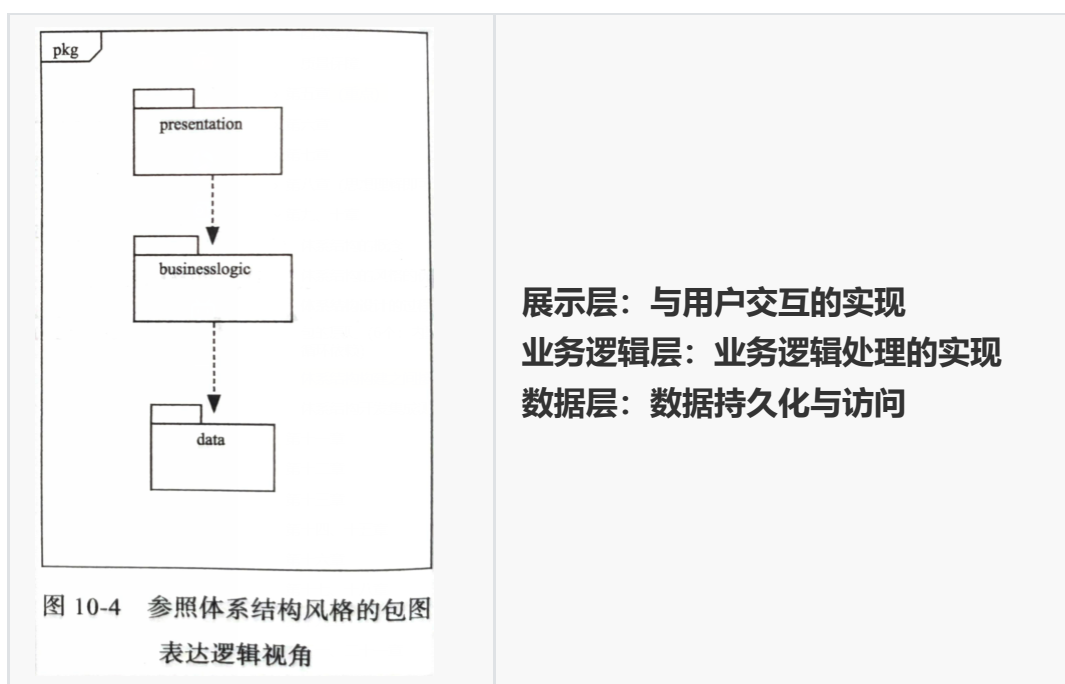
- 体系结构设计是软件非功能性的实现，而详细设计主要是软件功能性的实现。

1. 分析关键需求和项目约束

- 分析用例文档和需求规格说明书（包含需求规格和项目约束）。注意既要考虑功能性需求，又要考虑非功能性需求，甚至很大意义上体系结构设计是为了满足**非功能性需求**
- 落实所有需求，并且必须满足项目的约束条件

2. 通过选择体系结构风格

- 信息系统——MVC或分层
- 基于Web开发——MVC
- 对实时性能要求不高，对修改和灵活性要求高，不希望使用web技术——分层
- 例：分层体系结构风格



3. 进行软件体系结构逻辑（抽象）设计——生成逻辑包图

- 根据概要功能需求和体系结构风格建立初始设计
 - 相似功能整合

表 10-1 连锁商店管理系统的初步概要功能设计

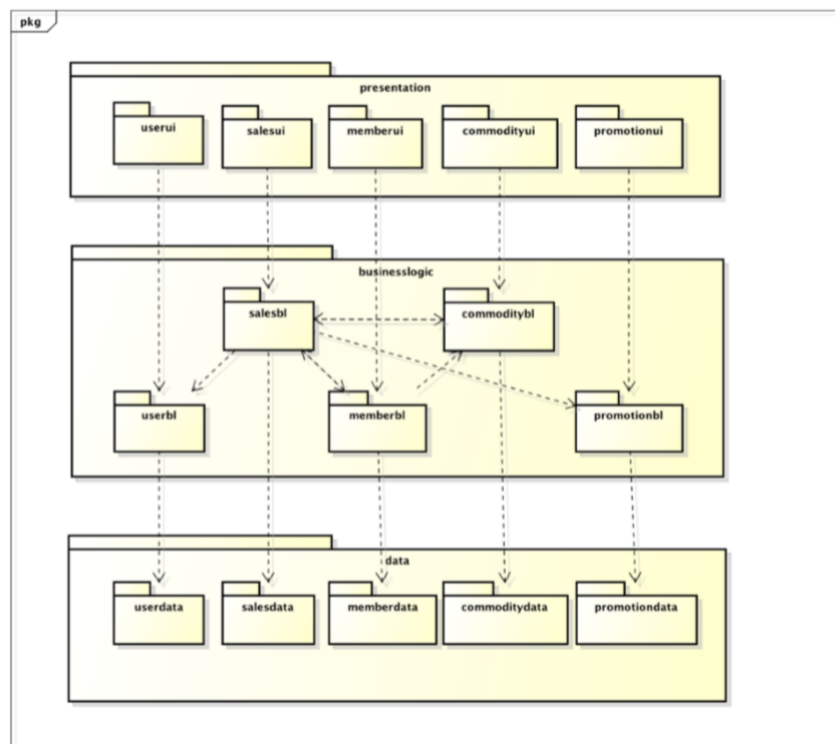
功 能	对应逻辑包
销售	salesui, salesbl, salesdata
库存	commodityui, commoditybl, commoditydata
会员	memberui, memberbl, memberdata
销售策略	promotionui, promotionbl, promotiondata
调整用户	userui, userbl, userdata

■ 分析不相似功能是否部分使用相同信息或行为

表 10-2 连锁商店管理系统的改进概要功能设计

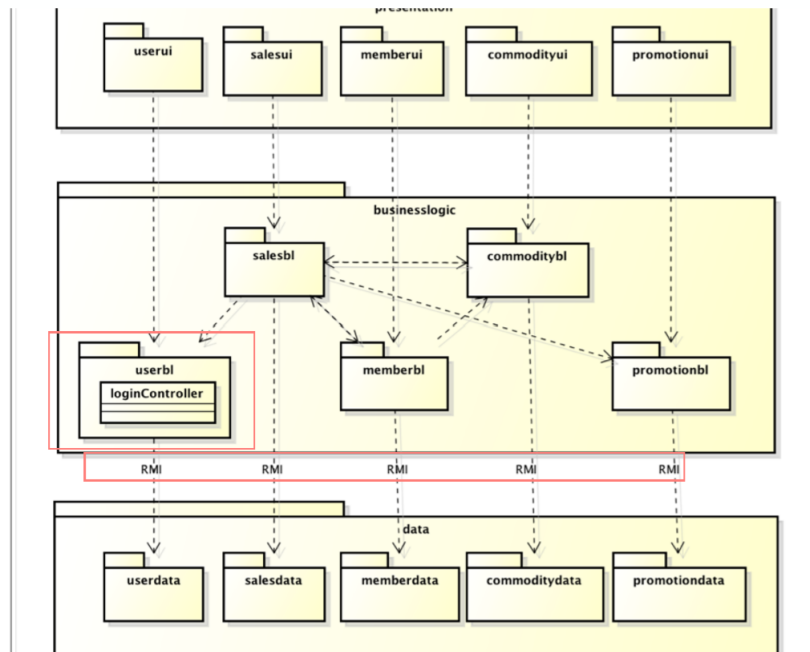
功 能	对应逻辑包
销售	salesui, salesbl, salesdata commoditybl, commoditydata memberbl, memberdata promotionbl, promotiondata userbl, userdata
库存	commodityui, commoditybl, commoditydata salesbl, salesdata
会员	memberui, memberbl, memberdata salesbl, salesdata commoditybl, commoditydata
销售策略	promotionui, promotionbl, promotiondata
调整用户	userui, userbl, userdata

■ 生成逻辑包图



○ 使用非功能性需求与项目约束评价和改进初始设计

- 比如对用户登陆与验证功能的安全需求以及万罗分布约束（将数据层部署在服务器端，则交互无法通过程序调用，而需要使用RMI）

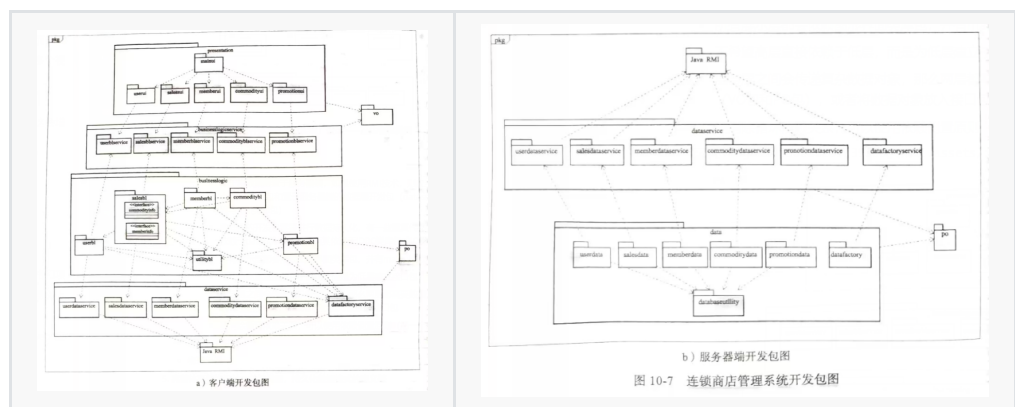


4. 依赖逻辑设计进行软件体系结构（实现）设计——产生物理类图

○ 开发包（构件）设计

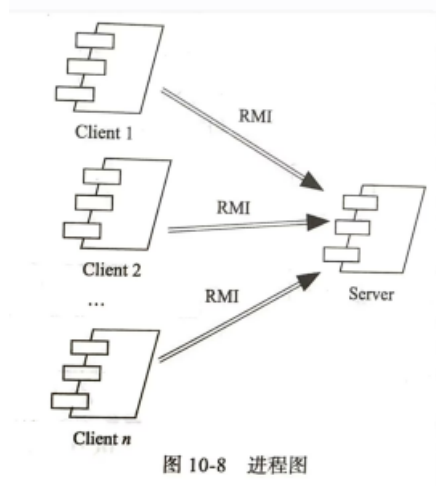
■ 一些考量点

- 由于将数据层置于服务器端，如果使用RMI技术，则应当将数据层的开发包分解为置于客户端的dataservice包和置于服务器端的数据层开发包（dataservice包和服务端的数据层开发包都依赖与RMI类库包）
- 数据层的开发包需要进行数据持久化，有一些重复代码可以独立为一个包，所有数据层开发包都依赖于databaseutility，而其依赖于JDBC类库包或IO类库包
- 所有展示层都会依赖图形界面类库包，并有mainui包负责整个页面之间的跳转逻辑
- 不希望高层直接依赖于低层，而是为低层建立接口包，让上下两层都依赖于接口包
- 层与层之间会传递复杂数据对象，那么两层都需要数据对象声明，把数据对象声明独立为开发包（VO包和PO包），或者将数据对象放入接口包
- 要消除循环依赖关系，可以使用依赖倒置原则（把一个包被依赖部分的接口放到另外一个包）



运行时的进程

- 表明运行时的进程，以及各进程之间如何通信



物理部署

上头的最后三步其实是包的设计过程，可以结合包的六个原则一起看

5. 完善体系结构设计：关键类图，持久化数据格式的定义等
6. 添加构件接口：包、重要文件的创建，定义接口
7. 迭代过程3-6

包的原则

1. 粒度：包的内聚性原则

- 包的内聚性原则可以帮助开发者决定如何将类划分到包中，依赖事实：至少已经存在一些类，并且它们之间的相互关系也已经确定，从“自底向上”的观点对类进行划分

1.1 重用发布等价原则 (REP)

- 重用的粒度就是发布的粒度
- 为重用器分组组件 (类)
- 单个类通常是不可重用的：一般是几个协作类组成一个包
- 包中的类应构成可重用和可发布的模块：模块提供要一致的功能
- 减少重新使用这者的工作
- **和关联类一起发布，而不是单独进行发布**

1.2 共同封闭原则 (CCP)

- 包中所有类对于同一类性质的变化应该是共同封闭的，一个变化如果对一个包产生影响，则对该包中所有类产生影响，而对于其他包不造成任何影响（只对可预测的变更有作用，不可预测的变更会为系统带来极大的破坏）
- 最小化修改对程序员的影响
- 包尽量大，这一点上与CRP是互斥的
- 方法：
 - 将具有相似闭包的类分组
 - 面向可以预期的变更封闭包
 - 将更改限制为几个软件包
 - 降低包的发布频率

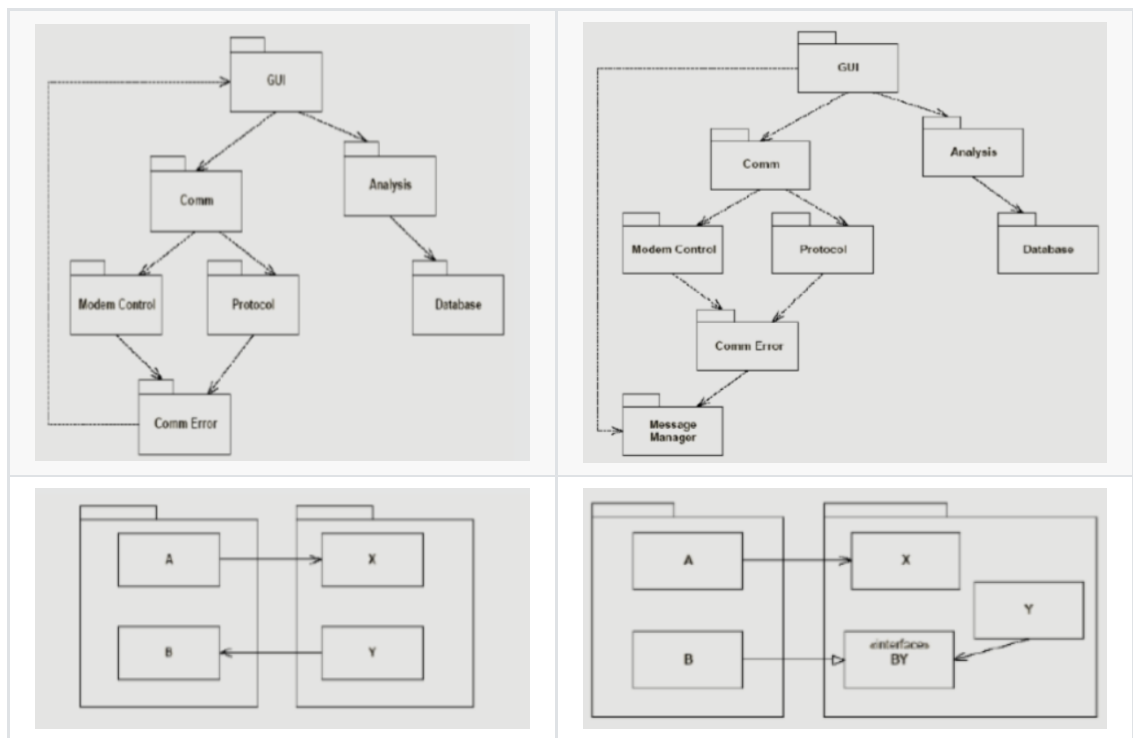
1.3 共同重用原则 (CRP)

- 一个包中的所有类应该是能够重用的。
- 根据常见重用类进行分组：避免给用户不必要的依赖
- 遵循CRP通常会导致软件包的拆分：获得更多、更小、更专注的包
- 减少重新使用者的工作
- 包尽可能小，和CCP互斥

2. 稳定：包的耦合性原则

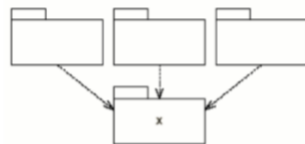
2.1 无环依赖原则 (ADP)

- 在包的依赖中不允许存在环，必须是有向无环图

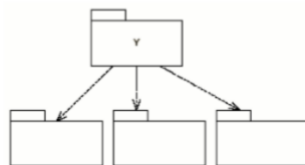


2.2 稳定依赖原则 (SDP)

- 朝着稳定（别人修改不影响我）的方向进行依赖
 - Ca: 输入耦合度，包外部依赖本包的类个数
 - Ce: 输出耦合度，包内部依赖于包外部的类的个数
 - 不稳定性: $(I = \frac{Ce}{Ce+Ca})$, I 越小越稳定
 - X is stable



- Y is instable



注意箭头是依赖

2.3 稳定抽象原则 (SAP)

- 包的抽象程度应该和其稳定程度一致
- 稳定的包应该是抽象的包，不稳定的包应该是具体的包
 - Na: 包中抽象类个数
 - Nc: 包中所有类个数
 - 抽象度 $(A = \frac{Na}{Nc})$

体系结构构建之间接口的定义

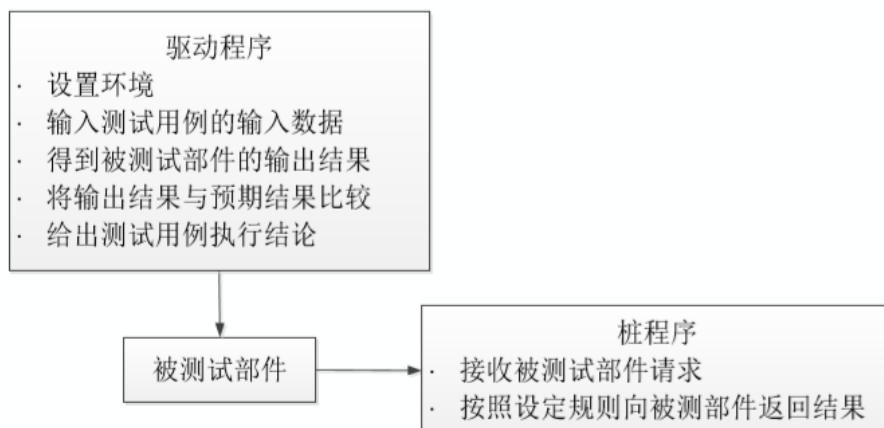
重点，逻辑层接口看需求、数据层接口被逻辑层调用增删改查

1. 确定模块对外接口
2. 编写接口规范

提供的服务（供接口）		
LoginController.login	语法	public ResultMessage login(long id, String password);
	前置条件	用户输入有效的用户编号和密码。user 成员变量初始化为 null。
	后置条件	根据 id 查找是否存在相应的用户，根据用户信息验证其密码正确性，如果通过验证记录当前用户。
需要的服务（需接口）		
服务名	服务	
UserDataService.find	根据 ID 进行查找用户信息	

体系结构开发集成测试用例

题目都差不多



编写驱动程序，在桩程序帮助下进行集成测试

- Stub：为了完成程序的编译和连接而使用的暂时代码
 - 用来替换某些模块，一般和所替代的模块有相同的接口，并且模拟地实现了模块的行为
 - 使用场景：一般通过本地化的数据层对象作为stub测试，降低RMI测试带来的复杂度
 - 桩模拟的是下层，测试上层
 -
- Driver：驱动一个测试程序，就是驱动一个Service行为
 - 驱动模拟的是上层，测试下层

```
public interface SalesBLSERVICE _Stub implements SalesBLSERVICE {
    String commodityName;
    double commodityPrice;
    double commodityDiscount;
    CommodityPromotion promotion;
    String memberName;
    int memberPoint;
    double total;
    double change;

    public SalesBLSERVICE _Stub(String cn, double cp, double cd, CommodityPromotion p, String mn, int mp, double t, double c){
        commodityName = cn;
        commodityPrice = cp;
        commodityDiscount = cd;
        promotion = p;
        memberName = mn;
        memberPoint = mp;
        total = t;
        change = c;
    }
    //销售界面得到商品和商品促销的信息
    public CommodityVO getCommodityById(int id){
        return new CommodityVO(name, price, commodityDiscount);
    }
    public ArrayList<CommodityPromotionVO> getCommodityPromotionListById(int commodityID){
        ArrayList<CommodityPromotionVO> commodityPromotionList = new ArrayList<CommodityPromotionVO>();
        commodityPromotionList.add(new CommodityPromotionVO(promotion));
        return commodityPromotionList;
    }
}
```

Stub使用

```
public class SalesView{
    .....
    SalesBLSERVICE salesBl = new SalesBLSERVICE _Stub ("Cup",10.0,2.0, new CommodityPromotion(), "Karen", 500, 57.0, 3.0);
    .....
}
```

Driver定义

```
public class SalesBLSERVICE _Driver{
    public void drive(SalesBLSERVICE salesBLSERVICE){
        ResultMessage result = salesBLSERVICE. addMember(0911024);
        If(result== ResultMessage.Exist) System.out.println("Member exists\n");
        .....
        salesBLSERVICE.endSales();
    }
}
```

Driver使用

```
public class Client{
    public static void main(String[] args){
        SalesBLSERVICE salesController = new SalesController();
        SalesBLSERVICE _Driver driver = new SalesBLSERVICE _Driver (salesController);
        driver.drive(salesController);
    }
}
```

第十一章

可用性

- 可用性不是用户界面的单一维属性

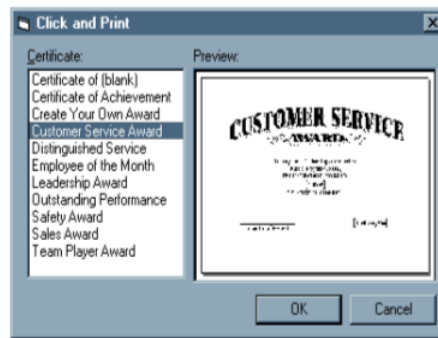
性，它包含以下维度

- 易学性：新手容易学习，能够很快使用系统
- 效率：熟练的用户能高效使用
- 易记性：使用过软件系统的用户，能够有效记忆或快速重新学会使用该系统
- 出错率：几乎没有错误，可以从错误中快速恢复
- 主观满意度：让用户有良好的体验

人机交互原则

例子xx违反了哪些条界面设计原则

1. 简洁设计（ 7 ± 2 原则）：文字不如图形
2. 一致性设计：实际模型与人的精神模型一致，如下图



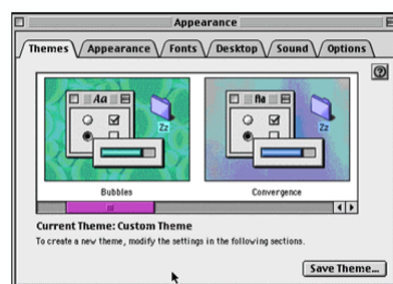
ok和cancel的位置不符合用户心智

3. 低出错率设计：包括不适当的菜单功能灰色屏蔽、检查输入，出现错误的时候帮助用户修正错误，而不是重新填写，每一个输入都尽快的显示问题
4. 易记性设计：减少短期记忆的负担；逐层递进显示信息；直观快捷方式；设置有意义的默认值
5. 可视化设计：
 - 按照任务模型设计界面隐喻，同时不要把软件系统的内部构造机制暴露给用户



功能和场景结合起来，而不是完全要全新选择，避免用户做出不当的行为

- 可视化设计还应该基于界面隐喻，尽可能地把功能和任务细节表现出来



左侧选择字体没有更加细节的展示，而右侧则展示了其中的细节问题

精神模型

- 精神模型是指用户进行人机交互时头脑中的任务模型，依据精神模型可以进行隐喻设计
 - 隐喻又被称为视觉隐喻，是视觉上的图像，但会被用户映射为业务事物。用户在识别图像时，会依据隐喻将控件功能与已知的熟悉事物联系起来，形成任务模型
 - 隐喻本质上是在用户已有知识的基础上建立一组新的知识，实现界面视觉提示和系统功能之间的知觉联系

差异性

- **新手用户：是对业务不熟悉的人。**为其设计系统时要关注易学性，进行业务导航，尽量避免出错。如果一个系统大多数用户是新手用户，整个系统的人机交互设计都要**侧重易学性**
- **专家用户：能够熟练操作计算机完成业务任务的人**，一般是长时间使用软件系统并且计算机操作技能熟练的人。为专家用户设计系统时，要关注效率。如果一个系统的大多数用户都是专家用户，整个系统的人机交互设计都要**侧重效率**
- 熟练用户：位于两种之间，设计时要进行易学性和效率的折中
- 好的人机交互应该为不同用户群体提供差异化的交互机制
 - 既为新手用户提供易学性高的人机交互机制（如图形界面）
 - 又为专家用户提供效率高的人机交互机制（如命令行、快捷方式、热键等）

导航、反馈、协作式设计

违反什么例子、适用什么原则

- 导航
 - 导航是为了给用户提供一个很好的完成任务的入口，好的导航保证这个入口很符合人的精神模型。
 - 全局导航按照任务模型将软件产品的功能组织起来，并区分不同的重要性和主题提供给不同的用户
 - 控件包括窗口、菜单、列表、快捷方式、热键等等。
 - 局部导航通过安排布局细节，制造视觉上的线索来提供导航
- 反馈
 - 反馈能够避免用户进行错误的操作，让用户明确自己的等待过程的长度。（视听）目的是提示用户交互行为的结果，但是不能打断用户的意识流
 - 一般使用等待动画降低等待感，或者增加进度条

- 总结
 - 用户喜欢较短的响应时间；
 - 较长的响应时间(>15秒)具有破坏性；
 - 用户会根据响应时间的变化调整自己的工作方式；
 - 较短的响应时间导致了较短的用户思考时间；
 - 较快的节奏可能会提高效率，但也会增加出错率；
 - 时间情况
 1. 打字、光标移动、鼠标定位：50 ~ 150 毫秒
 2. 简单频繁的任务：1 秒
 3. 普通的任务：2 ~ 4 秒
 4. 复杂的任务：8 ~ 12 秒
 - 响应时间适度的变化是可接受的；
 - 意外延迟可能具有破坏性；
 - **经验测试**有助于设置适当的响应时间
- 协作式设计
 - 人和计算机是人机交互的方法，其中人的因素是比较固定的，一定时期内不会发生大的变化，所以要让二者交互顺畅，就需要让计算机更多地适应人的因素，这也是人机交互设计以用户为中心的根本原因。
 - 这种调整计算机因素以更好地适应并帮助用户的设计方式被称为协作式设计

第十二章

详细设计的出发点

- 软件详细设计是在软件体系结构设计之后进行的
- 其出发点是：需求开发的结果（需求规格说明文档和需求分析模型）以及软件体系结构的结果（软件体系结构设计方案与原型）
- 其目的是实现所有功能性需求和非功能性需求

职责分配

- 职责是执行任务（操作职责）或维护某些数据（数据职责）的义务
 - 行为职责通常由行为来履行
 - 数据职责通常由属性来完成

- 可能会设计到类之间的协作
- 类的职责主要有属性职责和方法职责
- 在类的表示中，通过类的成员变量和成员方法来表示各个类的职责
- 职责的分配遵循GRASP原则（一般职责分配软件模式）
 - 特点：
 - 低耦合：分配一个职责要保证低耦合度
 - 高聚合：分配一个职责的时候要保持类的高聚合度
 - 拇指原则：当有替代设计选择时，请仔细研究替代方案的内聚和耦合影响，以及未来可能对替代方案的演变压力。选择具有良好内聚性、耦合性和稳定性的替代方案。
 - 信息专家：将一个职责分配给专家——拥有履行职责所必须的信息的类
 - 统一提供对外的接口，而不是某一个具体的接口，数据和行为不要分开
 - 创建者：创建规则在后面
 - 如果有以下情况，则由创建者分配 B 类创建 A 类实例的职责：
 1. B 聚合了 A 对象
 2. B 包含了 A 对象
 3. B 记录了 A 的实例
 4. B 要经常使用 A 对象
 5. 当 A 的实例被创建，B 具有传递给 A 的初始化数据（也就是 B 是创建 A 的实例这项任务的信息专家）
 6. 在有选择的地方，更喜欢 B 聚合或包含 A 对象
 - 控制者：控制规则在后面（避免大多数信息由一个类发出、组件相对较小、行为职责和数据绑定、职责单一）

表 12-4 对象创建者

优 先	场 景	创建地点	创建时机	备 注
高 ↑ ↓ 低	唯一属于某个整体的密不可分的一部分（组合关系）	整体对象的属性定义和构造方法	整体对象的创建	例如，销售的业务逻辑对象由销售页面对象创建
	被某一对象记录和管理（单向被关联）	关联对象的方法	业务方法的执行中对象的生命周期起始点	例如，连接池管理对象需要负责创建连接池对象
	创建所需的数据被某个对象所持有	持有数据的对象的业务方法	业务方法的执行中	也可以考虑在此持有数据对象不了解创建时机时，由别的对象创建，由它来初始化
	被某个整体包含（聚合关系）	整体对象的业务方法（非构造方法）	业务方法的执行中	如果某个对象有多个关联，优先选择聚合关联的整体对象。如果某个对象有多个聚合关联的整体对象，则考查整体对象的高内聚和低耦合来决定由谁创建
	其他			通过高内聚和低耦合来决定由谁创建

- 通过职责建立静态模型，使用分析类图表示对象的职责

协作

- 程序中的对象互相协作，完成一个较大的职责。
- 通过协作建立动态设计模型，使用顺序图表示对象之间的协作

控制风格

1. 集中式控制风格：一个额外的控制器记录系统行为的所有逻辑，容易找到做出决定的位置，但是控制器比较庞大，复杂且难以理解，都是控制器调用他者
2. 委托式控制风格：做出决策的对象不只有一个，职责的分解决定了控制对象的层次。
3. 分散式控制风格：无法找到明确的控制对象，靠对象自治的方式实现大的职责。系统行为的逻辑通过对象网络**广泛传播**，隐藏信息很难，内聚性比较差

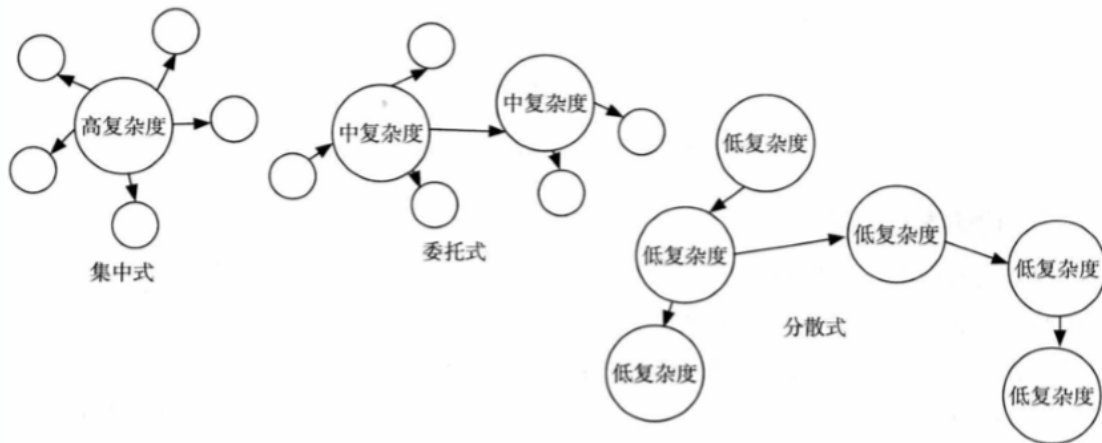


图 12-13 控制风格

给定分析类图、系统顺序图和设计因素描述

- 建立设计类图
- 或者详细顺序图

协作的测试

协作的测试

- Mock Object: 类似于Stub，但是更加简洁，要求其不用测试就能保证质量，用它来代替某一层的程序

```

public class TotalIntegrationTester {
    @Test
    public void testTotal () {
        MockCommodity commodity1 = new MockCommodity (50);
        MockCommodity commodity2 = new MockCommodity (40);
        SalesLineItem salesLineItem1 =
            new SalesLineItem (commodity1, 2);
        SalesLineItem salesLineItem2
            = new SalesLineItem (commodity2, 3);

        Sales sale=new Sales();
        sale.addSalesLineItem(salesLineItem1);
        sale.addSalesLineItem(salesLineItem2);

        assertEquals (220, sale.total ());
    }
}

public class MockCommodity extends Commodity{
    double price;
    public MockCommodity ( double p){
        price=p;
    }
    public getPrice () {
        return price;
    }
}

```

第十三章

耦合与内聚

耦合与内聚：对例子，说明它们之间的耦合程度和内聚，给出理由

- 内聚：表达的是一个模块内部的联系的紧密型：包括信息内聚、功能内聚、通信内聚、过程内聚、逻辑内聚和偶然内聚

类 型	内 聚 性	解 释	例 子
偶然内聚	最低 ↑ ↓ 最高	模块执行多个完全不相关的操作	把下列方法放在一个模块中：修车、烤面包、遛狗、看电影
逻辑内聚		模块执行一系列相关操作，每个操作的调用由其他模块来决定	把下列方法放在一个模块中：开车去、坐火车去、坐飞机去
时间内聚		模块执行一系列与时间有关的操作	把下列方法放在一个模块中：起床、刷牙、洗脸、吃早餐
过程内聚		模块执行一些与步骤顺序有关的操作	把下列方法放在一个模块中：守门员传球给后卫、后卫传球给中场球员、中场球员传球给前锋、前锋射门
通信内聚		模块执行一系列与步骤有关的操作，并且这些操作在相同的数据上进行	把下列方法放在一个模块中：查书的名字、查书的作者、查书的出版商
功能内聚		模块只执行一个操作或达到一个单一目的	下列内容都作为独立模块：计算平方根、决定最短路径、压缩数据
信息内聚		模块进行许多操作，各个都有各自的入口点，每个操作的代码相对独立，而且所有操作都在相同的数据结构上完成	比如数据结构中的栈，它包含相应的数据和操作。所有的操作都是针对相同的数据结构

- 5. 显然，印记耦合可以实现更多的数据耦合
- 6. 控制耦合比数据耦合还耦合更多：信息隐藏

信息隐藏

基本思想

- 每个模块都隐藏了一个重要设计决策。每个模块都承担一定的职责，对外表现为一份契约，并在契约之下隐藏只有这个模块知道的设计决策或者秘密，决策实现细节只有该模块的组成部分才知道。
- 抽象出来的是接口，隐藏的是实现
- 秘密：
 - 主要秘密：主要秘密描述的是这个模块所要实现的**用户需求**。是设计者对用户需求的实现的一次职责分配。有了这个描述以后，我们可以利用它检查我们是否完成所有的用户需求，还可以利用它和需求优先级来决定开发的次序。
 - 次要秘密：次要秘密描述的是这个模块在实现职责时候所涉及的具体的**实现细节**。包括数据结构，算法，硬件平台等信息。

两种常见的信息隐藏决策

对例子xx，说明其信息隐藏程度好坏

1. **根据需求分配的职责**，因为实践表明，需求是经常变化的，频率和幅度都很大；
2. **内部实现机制**，常见的变化主题包括硬件依赖，输入输出形式，非标准语言特征和库，负责的设计和实现，复杂的数据结构，复杂的逻辑，全局变量。数据大小限制等。

第十四、十五章

几个原则，根据原则评判代码好坏

1. **全局变量是被认为是有害的**
 1. 公共耦合是有风险，而且会增加潜在连接数量
2. **如果没有特殊要求，让代码清晰一点：**
 1. 让代码兼顾明确和可修改性
3. **避免重复**
 1. 面向接口编程，而不是重复地写逻辑上一致的代码
4. **面向接口编程**

5. 迪米特法则

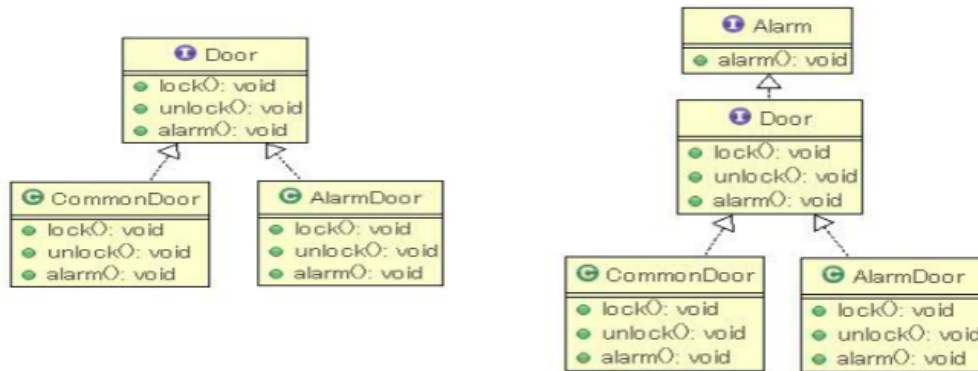
1. 你可以自己玩。(this)
2. 你可以玩自己的玩具，但不能拆开它们（自己的成员变量）
3. 你可以玩送给你的玩具。（方法）
4. 你可以玩自己制作的玩具。（自己创建的对象）
5. 强调的是不能出现 a.b.Methods 这类情况，不能人去动狗的腿，而是人命令狗，狗动腿

6. 接口隔离原则 (ISP) / 也叫接口最小化原则

1. 将大的接口拆解成几个小的接口。
2. 这样可以有效避免出现不必要的依赖。

7. 里氏替换原则 (LSP)

1. 所有派生类都必须可以替代其基类
2. 派生类的前置条件更弱，后置条件更强
3. 解决方案：在父类中添加方法或者函数，或者拆分接口



- 应该分为 Door 和 Alarm 两个接口，第一个 CommonDoor 不应该实现 alarm

8. 组合代替继承

1. MyStack 组合 Vector

9. 单一职责原则：一个类只能有一个改变的理由

10. 最小化类和成员的可访问性 (x 是可见)

- 是不是需要 public
- 类声明前没有 public，则包内可见
- 方法生命前没有 public，则包内可见
- public 修饰是全局可见

Access Specifier	Class	Package	Subclass	World
private	x			
None	x	x	x*	
protected	x	x	x**	
public	x	x	x	x

类和成员的可访问性

* Subclasses within the same package can also access members that lack access specifiers (default or package-private visibility). An additional requirement for access is that the subclasses must be loaded by the class loader that loaded the class containing the package-private members. Subclasses in a different package cannot access such package-private members.
 ** To reference a protected member, the accessing code must be contained either in the class that defines the protected member or in a subclass of that defining class. Subclass access is permitted without regard to the package location of the subclass.

11. 开放/封闭原则 (OCP)

1. 对扩展开放：模块的行为可以被扩展，比如新添加一个子类
2. 对修改关闭：模块中的源代码不应该被修改
3. RTTI（运行时类型信息违反了开闭原则 LSP），就是如果都有的话则抽象成基类的方法

12. 依赖倒置原则 (DIP)：高级模块不应依赖于低级模块：两者都应依赖抽象。



分析下图请说明：（1）它正确运用了哪条面向对象设计原则，请详细解释这条原则（5分）；（2）User类的内部是哪种层次的内聚？请详细描述这个层次内聚的特征（5分）；

重点看一下封装、OCP、DIP

第十六章

可修改性、可扩展性、灵活性

- （狭义）可修改性
 - ：涉及到大的场景的修改
 - 1. 对已有实现的修改
 - 2. 例如：修改现有促销策略

- 可扩展性 (DIP & OCP)
 1. 对新的实现的扩展
 2. 例如：增加一条新的促销策略（策略模式）
- 灵活性
 1. 对实现的动态配置
 2. 例如：动态修改更改某商品对应促销策略
- 它们的实现——将接口与实现分离
 1. 通过接口和实现该接口的类完成接口与实现的分离
 2. 通过子类继承父类，将父类的接口和子类的实现相分离

设计模式

策略模式 (ifelse)、迭代器模式，不考抽象模式

给定场景，应用设计模式并写代码

给出代码，要求用设计模式改写

1. 策略模式

- 在策略模式中，一个类的行为或其算法都可以在运行时改变。属**行为型模式**。
- 在策略模式中，我们创建表示各种策略的对象和一个行为随着策略对象改变而改变的 context 对象。策略独享改变 context 对象的执行算法。
- 设计原则与解释
 - 减少耦合：减少策略的使用类和策略的实现类直接的耦合——分解+组合而非继承+策略在策略类中提供
 - 依赖倒置：策略的使用类依赖的是策略的接口，而非策略的实现类
- 代码和实例看书本P269

2. 迭代器模式

- 在迭代器模式中，抽象了对遍历的操作：是否有下一个元素和得到下一个元素
- 迭代器模式只提供了对集合访问的方法，屏蔽了对结合修改的方法，这样将集合作为参数，就可以做到对集合“值传递”的效果
- 设计原则与解释：
 - 减少耦合：减少遍历的使用类和遍历的实现类直接的耦合
 - 依赖倒置：遍历的使用类依赖的是策略的接口，而非遍历的实现类
- 代码和实例看书本P279

第十七、十八章

名词解释

- **软件构造包含的活动**：软件构造除了核心的编程任务之外，还涉及到了详细设计（数据结构与算法设计）、单元测试、集成与集成测试以及其他活动
 - **重构**：修改软件系统的严谨方法，它在不改变代码外部表现的情况下改进其内部结构
 - **测试驱动开发**：测试驱动开发要求程序员在编写一段代码之前，优先完成该段代码的测试代码。并在编程中重复执行测试代码，以验证程序的正确性
 - **结对编程**：两个程序员挨着坐在一起，共同协作进行软件构造活动
-

给定代码段实例，对其进行改进或发现其中的问题

- 简洁性/可维护性
 - 使用数据结构消减复杂度判定
 - 控制结构
 - 变量使用
 - 语句处理
 - How to write unmaintainable code
 - 防御与错误处理
-

单元测试用例的设计

- 按照输入输出测试构造
- 为方法开发测试用例主要使用两种线索：
 - (1) 方法的规格；
 - 根据第一种线索，可以使用基于规格的测试技术开发测试用例，等价类划分和边界值分析是开发单元测试用例常用的黑盒测试方法。
 - (2) 方法代码的逻辑结构。
 - 根据第二种线索，可以使用基于代码的测试技术开发测试用例，对关键、复杂的代码使用路径覆盖，对复杂代码使用分支覆盖，简单情况使用语句覆盖。

- 在复杂类中，常常有着多变的状态，每次一个方法的执行改变了类状态时，都会给其他方法带来影响，也就是说复杂类的多个方法间是互相依赖的。
 - 所以，除了测试类的每一个方法之外，还要测试类不同方法之间的互相影响情况。
-
- 单元测试用例：输入，方法，预期输出

契约式设计

- 使用异常或断言的方式检查
 - 检查中间的过程
-

防御式编程：对输入进行检测

- - 防御式编程的基本思想是：在一个方法与其他方法、操作系统、硬件等外界环境交互时，不能确保外界都是正确的，所以要在外界发生错误时，保护方法内部不受损害。
 - 常见场景
 - 输入参数是否合法？
 - 用户输入是否有效？
 - 外部文件是否存在？
 - 对其他对象的引用是否为NULL？
 - 其他对象是否已初始化？
 - 其他对象的某个方法是否已执行？
 - 其他对象的返回值是否正确？
 - 数据库系统连接是否正常？
 - 网络连接是否正常？
 - 网络接收的信息是否有效？
 - 异常和断言都可以用来实现防御式编程，两种实现方式的差异与契约式设计的实现一样。

表驱动

- 用空间换时间，数据的保存一定要能够节省计算

第十九章

白盒测试和黑盒测试

掌握常用方法，进行优缺点比较

黑盒：等价类划分、边界值

白盒：语句覆盖、分支覆盖和路径覆盖

黑盒测试

- 将测试对象作为一个黑盒子，完全基于输入输出来判断测试对象的正确性
- 方法：

1. 等价类划分

- 等价类划分是把所有可能的输入数据，即程序的输入域划分为若干部分（子集），然后从每个子集中选取少数具有代表性的数据作为测试用例
- 有效等价类：是指对于程序的规格说明来说是合理的、有意义的输入数据的集合。利用有效等价类可检验程序是否实现了规格说明文档中所规定的功能和性能
- 设计时不仅要有有效等价类，也要有无效等价类，才能确保软件具有更高的可靠性

2. 边界值分析

- 边界值分析是等价类划分方法的补充，实践证明，错误最容易发生在各等价类的边界，而不是等价类的内部。
- 对等价类划分的补充，错误容易发生在各个等价类的边界上，而不是等价类的内部，因此针对边界情况设计测试用例，可以发现更多的缺陷。

3. 决策表

- 决策表是为**复杂逻辑判断**设计测试用例的技术。决策表示由条件声明、行动声明、规则选项和行动选项等四个象限组成的表格。

4. 状态转换

- 状态转换是针对复杂测试对象的测试技术。该类复杂测试对象对输入数据的反映是多样的，还需要依赖自身的状态才能决定。
 - 状态转换包含有效转换和无效转换，只有在复杂情况和可靠性要求较高的情况下才会为无效转换设计测试用例。
 - 状态图——状态转换表——测试用例
-

白盒测试

- 白盒测试是将测试对象看做透明的，不关心测试对象的规格，而是按照测试对象内部的程序结构来设计测试用例进行测试工作
- 方法：

1. 语句覆盖

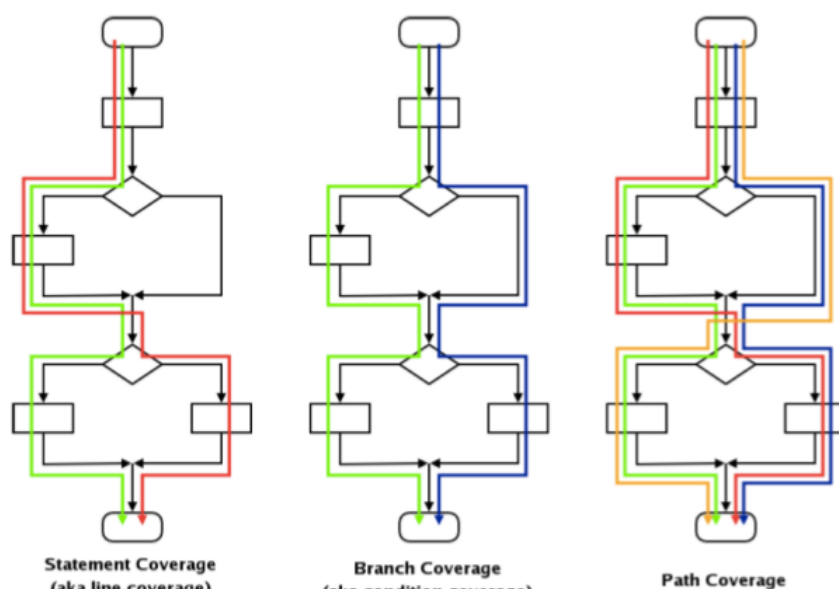
- 语句覆盖设计测试用例的标准时确保被测试对象每一行程序代码都至少执行一次

2. 路径覆盖

- 确保每条独立的执行路径都至少执行一次

3. 条件覆盖

- 确保每个判断结果都至少满足一次



给出一个场景，判断使用哪种测试方法，如何写（重点）

- 对给定场景和要求的测试方法，设计测试用例
 - 给出功能需求，则要求写功能测试用例
 - 给出设计图，则要求写集成测试用例，Stub and Driver
 - 给出方法的描述，则要求写单元测试用例，Mock Object
 - JUnit基本使用方法

第二十二、二十三章

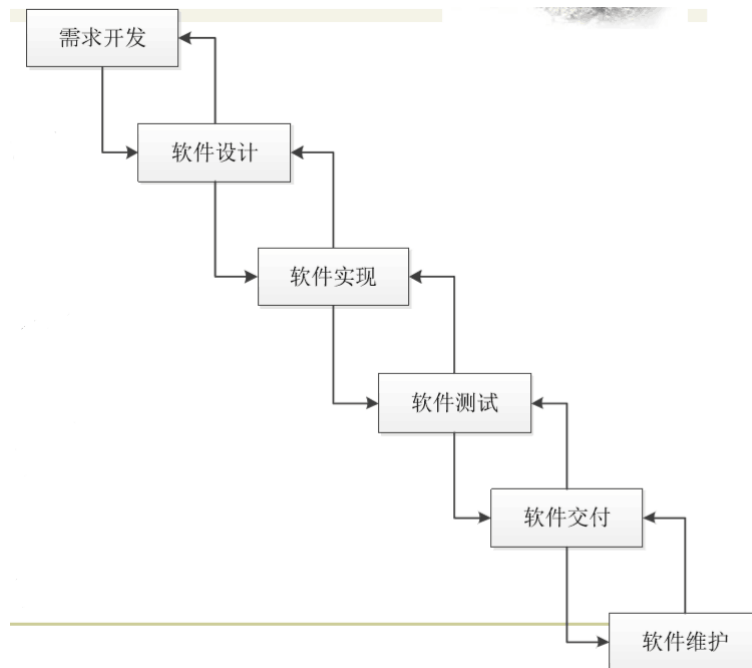
软件生命周期模型

要求、特征描述、优点、缺点

给定场景，判定适用的开发过程模型

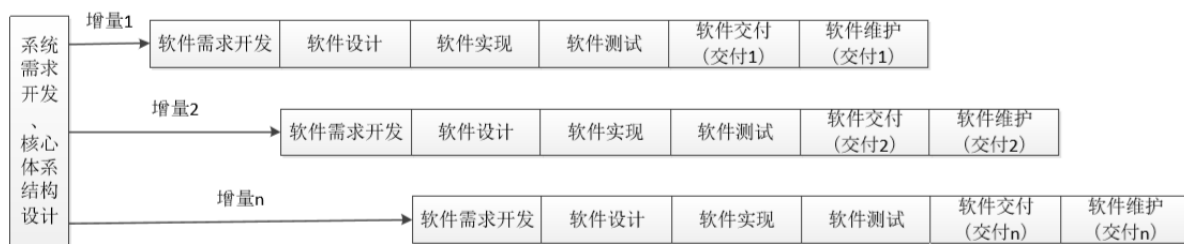
软件工程体系的知识域

瀑布模型



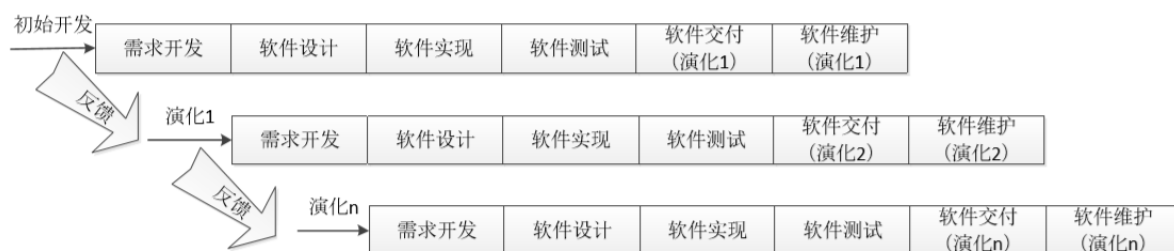
- 文档驱动，按照软件生命周期模型将软件开发活动组织为需求开发、软件设计、软件实现、软件测试、软件交付和软件维护行为，并且规定了它们自上而下、相互邻接的次序
- 适用于比较成熟，没有技术难点的软件
- 优点：
 - 明确的阶段划分，开发者关注点分离的方式更好地进行开发（对个人而言开发复杂度降低）
- 缺点：
 - 对文档的过高的期望具有局限性
 - 对开发活动的线性顺序假设具有局限性
 - 客户、用户的参与具有局限性：成功的项目开发需要客户、用户从始至终的参与，而不仅仅是一个阶段。
 - 里程碑粒度具有局限性：里程碑粒度过粗，基本丧失了“早发现缺陷早修复”这一思想

增量模型



- 需求驱动，在项目开始时，通过系统需求开发和核心体系结构设计活动完成项目对前景和范围的界定，然后再将后续开发活动组织为**多个迭代、并行的瀑布式开发模型**
- 使用于大规模软件系统开发
- 优点：
 - 迭代式开发更加符合软件开发的实践情况，具有更好的适用性；
 - 并行开发可以帮助缩短软件产品的开发时间；
 - 渐进交付可以加强用户反馈，降低开发风险
- 缺点：
 - 由于各个构件是逐渐并入已有的软件体系结构中的，所以加入构件必须不破坏已构造好的系统部分，这需要软件具备开放式的体系结构。
 - 增量交付模型需要一个完备、清晰的项目前景和范围以进行并发开发规划，但是在一些不稳定的领域，不确定性太多或者需求变化非常频繁，很难在项目开始就确定前景和范围。

演化模型



- 需求驱动，将软件开发活动组织为多个迭代、并行的瀑布式开发活动。
- 演化模型能够更好地应对需求变更，更适用于需求变更比较频繁或不确定性较多的领域
- 优点：
 - **使用了迭代式开发，具有更好的适用性**，尤其是其演化式迭代安排能够适用于那些需求变更比较频繁或不确定性较多的软件系统的开发；
 - 并行开发可以帮助**缩短**软件产品的开发时间；
 - 渐进交付可以**加强用户反馈**，降低开发风险。
- 缺点：

- 无法在项目早期阶段建立项目范围，所以项目的整体计划、进度调度、尤其是商务协商事宜无法准确把握；
- 后续迭代的开发活动是在前导迭代基础上进行修改和扩展的，这容易让后续迭代忽略设分析与设计工作，蜕变为构建-修复方式。

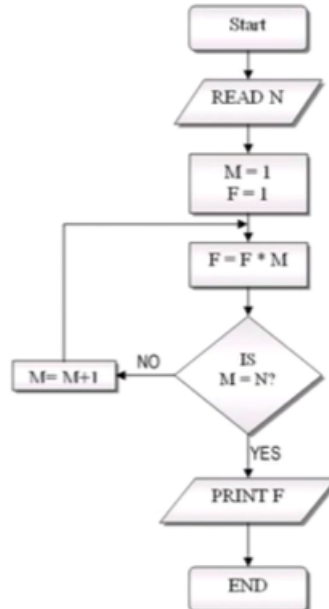
其他

- 构建修补模型——规模很小的软件
- 原型模型（需求驱动）——大量不确定的新颖领域进行开发活动组织
- 螺旋模型RUP模型（风险驱动）——高风险的大规模软件系统开发
- 敏捷模型（什么最有效果就做什么）——快速变化或者时间压力较大的项目。

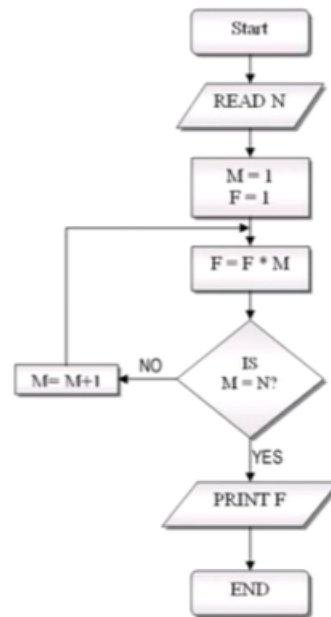
其他

- 给场景（找关键词）、使用原则、解决方式
- 代码的坏味道（注释、对齐、类的封装、、）
- 衡量圈图复杂度，两种方法
 - 这里涉及到两种计算方法

- 衡量圈复杂度的基本思路是计算程序中独立路径的最大数量。第一种计算方法是建立程序的流程图G，假设图的节点数为N，边数为E，那么复杂度 $V(G) = E - N + 2$ 。以右图为例，其节点数为8，边数为8，则程序的复杂度为2。通过直接分析右图也可以发现，它的确有两条路径。



- 还有一种简单的算法是直接计数程序中决策点的数量：
(1) 从1开始，一直往下通过程序。(2) 一旦遇到下列关键字，或者同类的词，就加1：if, while, repeat, for。(3) 给case语句中的每一种情况都加1。
- 例如，右图所描述的程序很明显只有一个DO-While语句，所以复杂度为2。



考试

- 数量：8-10道大题
- 第一题：三道名词解释（出不出大体的章节）